

pyALDIC-3D: An Open-Source, GUI-Based Stereo Digital Image Correlation Application with Built-In Calibration, Crack-Aware Surface Strain and Independent Result Verification

Zixiang Tong^a, Jin Yang^{a,b,*}

^a*Department of Aerospace Engineering and Engineering Mechanics, The University of Texas at Austin, Austin, TX 78712, USA*

^b*Texas Materials Institute, The University of Texas at Austin, Austin, TX 78712, USA*

Abstract

We present **pyALDIC-3D**, an open-source, cross-platform desktop application for stereo digital image correlation that covers the whole measurement chain — camera calibration, stereo and temporal correspondence, triangulation and finite-strain surface analysis — in a single tool. It is the first implementation of the Augmented Lagrangian DIC formulation for stereo outside MATLAB, and adds three capabilities rarely found together: quality-controlled built-in calibration; *crack-aware* strain, which measures discontinuous fields instead of smoothing across them; and an independent re-verification of every shipped displacement that exposes silent solver failures. Agreement with the published MATLAB reference implementation is 1.3/1.0/6.0 μm in $U/V/W$. pyALDIC-3D is BSD-3-Clause licensed and openly available at <https://github.com/zachtong/pyALDIC-3D>.

Keywords: stereo digital image correlation, 3D-DIC, camera calibration, discontinuous deformation, surface strain, open-source scientific software

*Corresponding author.

Email address: jin.yang@austin.utexas.edu (Jin Yang)

Required metadata

Current code version

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.1.0
C2	Permanent link to code / repository used for this code version	https://github.com/zachtong/pyALDIC-3D
C3	Code Ocean compute capsule	—
C4	Legal Code License	BSD 3-Clause
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python (version ≥ 3.10); PySide6; NumPy; SciPy; OpenCV; Numba; PyVista; matplotlib; pytest; al-dic (the pyALDIC 2D correlation engine, consumed as a pinned library)
C7	Compilation requirements, operating environments & dependencies	OS-independent (Windows, macOS, Linux). Install with <code>pip install al-dic-3d</code> , from the standalone Windows installer, or editable for development with <code>pip install -e ".[dev]"</code> . No compilation required; Numba JIT-compiles the strain and verification kernels at first invocation.
C8	If available, link to developer documentation / manual	https://github.com/zachtong/pyALDIC-3D/tree/main/docs/user-guide
C9	Support email for questions	zachtong@utexas.edu

Table 1: Code metadata for pyALDIC-3D v1.1.0. A permanent archive is deposited at Zenodo [1].

Current executable software version

Nr.	(Executable) software metadata description	Please fill in this column
S1	Current software version	v1.1.0
S2	Permanent link to executables of this version	https://pypi.org/project/al-dic-3d/1.1.0/ ; standalone Windows installer on the GitHub Releases page
S3	Legal Software License	BSD 3-Clause
S4	Computing platforms / Operating Systems	OS-independent (Windows, macOS, Linux)
S5	Installation requirements & dependencies	Python (version ≥ 3.10); PySide6; NumPy; SciPy; OpenCV; PyVista; al-dic (installed automatically by <code>pip install al-dic-3d</code>). The Windows installer requires no Python installation.
S6	If available, link to user manual	https://github.com/zachtong/pyALDIC-3D/tree/main/docs/user-guide
S7	Support email for questions	zachtong@utexas.edu

Table 2: Executable software metadata for pyALDIC-3D v1.1.0, installable from PyPI as `al-dic-3d`.

1. Motivation and significance

Two-dimensional digital image correlation (DIC) measures the in-plane motion of a nominally flat surface imaged by one camera [2, 3]. That model breaks whenever the specimen moves out of plane: rigid out-of-plane translation, bending, necking and any curved surface produce apparent in-plane strain a single camera cannot distinguish from real deformation. *Stereo*-DIC removes the ambiguity by imaging the surface with two calibrated cameras, matching each material point in both views and triangulating it, so shape, three-component displacement and true finite surface strain are recovered on curved, moving specimens [2, 4]. It is the configuration most experimental campaigns need, and the one the community’s good-practice guidance is written around [5, 6, 7].

The price is a substantially longer measurement chain. A 2D analysis needs an image sequence and a region of interest; a stereo analysis additionally needs a metric camera calibration, a policy for finding correspondence *across* cameras as well as through time, a triangulation step, and a surface-strain formulation on a curved manifold rather than a pixel grid. Every link is a place where a tool can stop short, and open-source tools stop at different points (Table 3). **MultiDIC** [8] and **DuoDIC** [9] are capable MATLAB toolboxes but need a MATLAB license and delegate correlation to Ncorr [10]; **ADIC3D** [11] is a compact MATLAB reference implementation rather than an application; **DICe** [12, 13] and **OpenCorr** [14] are C++ codebases built from source; and **iCorrVision-3D** [15], the closest open-source peer, is an integrated Python stereo-DIC application with its own calibration module. Commercial systems [16, 17, 18] are polished but closed, and their algorithms are not publicly documented.

Two capabilities are missing across that landscape. First, *discontinuity*: a crack, shear band or debond makes the surface field discontinuous, yet a strain gauge whose neighbourhood straddles the discontinuity averages the two sides and reports a smooth ramp where there is a jump. Second, *verification*: DIC solvers routinely convert per-point failures into plausible finite numbers — bad subsets are replaced by interpolated neighbours, a global regularizer returns a field finite everywhere, and incremental composition fills gaps by nearest neighbour — so a pipeline that only checks finiteness cannot tell a converged solution from a frozen one, and ships both as valid.

pyALDIC-3D is an independent application (its own project format, workflow and 3D visualization layer) that consumes the pyALDIC 2D correlation engine [19] as a pinned, read-only library, and takes the MATLAB 3D-Stereo-ALDIC implementation of Tong et al. [20] as its mathematical reference. It is explicitly *not* a “3D mode” inside the 2D application. Its contributions are:

- (i) **A complete, accessible stereo pipeline.** Calibration through 3D strain in one cross-platform, BSD-3-Clause, pip-installable desktop application, with a one-click Windows installer, a headless CLI and eight localized interfaces. Calibration is built in — four board families including a coded circular-target detector developed for this work, six import formats, bundle adjustment and quantitative QC. To our knowledge it is also the first implementation of the Augmented Lagrangian (local–global) DIC formulation [21] for stereo outside MATLAB.
- (ii) **Crack-aware stereo DIC.** A thin barrier cut into the region of interest

along the discontinuity propagates through the whole chain: it cuts the mesh, excludes cross-barrier neighbours from the strain plane fit, and blanks the rendering, so a discontinuous field is measured rather than smoothed across (Section 2.2.3).

- (iii) **Verification by construction.** Every shipped cumulative displacement is re-verified by an *independent* zero-normalized correlation criterion, evaluated against the reference frame rather than against the solver’s own objective, which catches silent failures that a conventional pipeline reports as valid (Section 2.2.4).
- (iv) **Engineering for real campaigns.** Millimetre-native output, lazy image streaming that holds a 150-frame $\times$ 12 Mpx run to 5.68 GB of resident memory, partial results kept on cancellation, single-file session persistence, and Numba-accelerated kernels.

We are explicit about prior art: the AL-DIC algorithm is due to Yang and Bhattacharya [21], its adaptive-mesh treatment of complex geometry to Yang et al. [22], and the stereo extension — including the plane-fit surface-strain formulation reproduced here — was developed and validated in MATLAB by Tong et al. [20]. pyALDIC-3D is a from-scratch Python application that takes that work as its mathematical model and contributes the calibration stage, the crack-aware chain, the verification gate, the interface and the distribution.

2. Software description

2.1. Software architecture

pyALDIC-3D is a Qt-free computational core wrapped by a desktop interface (Fig. 1). The core is seven packages — `calibration`, `sequence`, `matching`, `reconstruct`, `strain3d`, `export`, `project` — none importing Qt, so the pipeline is scriptable and runs headless on a server; `viz3d`, `gui` and `i18n` add the interactive layer, and one console script exposes `run`, `gui` and `calibrate`. The 2D AL-DIC engine enters only through `matching`, as an externally pinned dependency rather than a vendored copy. The implementation is about 28,000 lines across 136 modules, covered by 687 tests run in continuous integration on Linux, macOS and Windows for Python 3.10 and 3.12.

The central architectural commitment is that stereo/temporal correspondence is a *pluggable* component rather than a hard-coded pipeline. A `CorrespondenceStrategy` protocol maps (sequence, calibrated rig, reference mesh) to a `CorrespondenceSet`: per-frame, per-node pixel positions in both cameras, with a correlation

Name	Refs	Language	GUI	Built-in calib.	Global	Discontinuity	Verification	Distribution
MultiDIC	[8]	MATLAB	●	●	○	○	●	Source
DuoDIC	[9]	MATLAB	●	●	○	○	●	Source
ADIC3D	[11]	MATLAB	○	○	○	○	○	Source
DICe	[12, 13]	C++	●	●	●	●	○	Source
OpenCorr	[14]	C++	●	●	○	○	○	Source
iCorrVision-3D	[15]	Python	●	●	○	○	○	Source
3D-Stereo-ALDIC	[20]	MATLAB	●	○	●	●	○	Source
pyALDIC-3D (this work)	[1]	Python	●	●	●	●	●	PyPI

● yes ○ no ● partial.

Feature data were compiled from public project documentation and source repositories (as of July 2026) and may contain errors.

Table 3: Representative open-source stereo/3D-DIC software. “Built-in calib.” = metric stereo calibration inside the tool rather than delegated to an external toolbox; “Global” = a global (finite-element / regularized) formulation alongside the local subset solve; “Discontinuity” = explicit crack/hole handling in *both* correlation and strain; “Verification” = an independent re-check of the delivered field beyond the solver’s own residual. MultiDIC, DuoDIC and ADIC3D require a MATLAB license.

quality and a provenance flag for every value. Everything downstream consumes only that object and is forbidden by an architecture test from importing any concrete strategy, so adding a fourth strategy (Section 2.2.2) changes nothing downstream.

2.2. Software functionalities

2.2.1. Stereo calibration

Calibration has no 2D equivalent and is mandatory: without intrinsics and relative pose, matched pixels cannot be triangulated. pyALDIC-3D performs it internally, following Zhang’s planar-target method [23] with a Brown–Conrady distortion model, on OpenCV [24]. Four board families are detected — chessboard, ChArUco, circle grid, and a coded circular-dot target whose detector was developed for this work: three ring fiducials establish board identity and orientation so partial views index correctly, edge-clipped dots are recovered by an elliptical arc fit with sub-pixel rim refinement rather than centroided, and the projected-circle centroid bias is corrected analytically. Per-camera and stereo geometry are then solved,

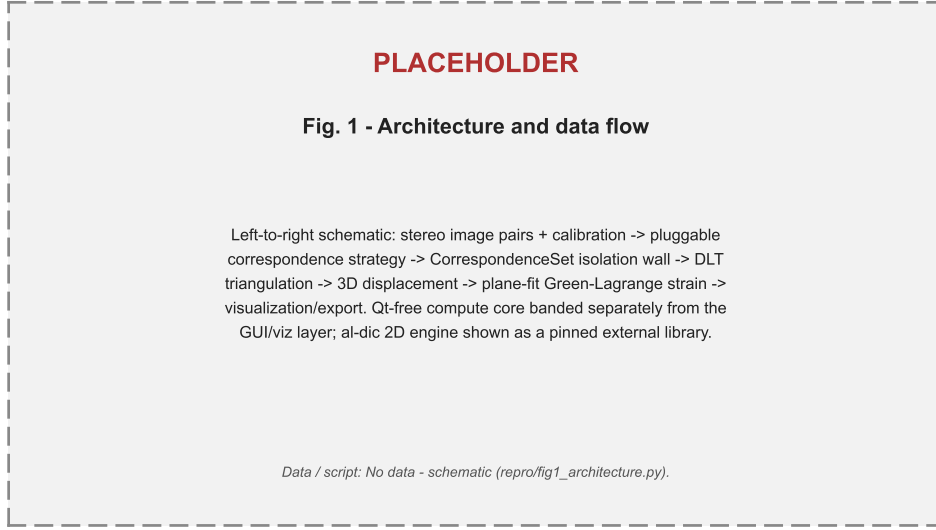


Figure 1: pyALDIC-3D architecture and data flow. Stereo image pairs and a calibration enter at the left; matching resolves correspondence through one of three interchangeable strategies and emits a single `CorrespondenceSet`. That object is an isolation wall: `reconstruct`, `strain3d`, `viz3d` and `export` consume only it, never a concrete strategy. The upper band is the Qt-free compute core, scriptable without the interface; the lower band is the desktop application. The pyALDIC 2D engine is a pinned external dependency of matching alone.

with an optional bundle adjustment that applies a robust per-point loss, exploits views seen by only one camera, and can optimize the board's own shape to absorb printing error; poor pairs are rejected against a user-set threshold and reported individually.

Every quantity needed to judge the result is displayed: stereo reprojection RMS and epipolar residual, baseline, pairs used versus total, image-plane coverage, board-tilt range, and the RMS change contributed by bundle adjustment. Following the iDICs position that a low reprojection residual is necessary but not sufficient [5], a separate *verification* pass measures a board of known geometry with the finished calibration and reports the recovered pitch, scale error and plane RMS in millimetres — a direct check on absolute metric scale (Fig. 2) — and a leave-*p*-out jackknife reports how sensitive the solution is to the choice of views. An existing calibration can instead be imported in one of six formats (DICE, MatchID, MATLAB `stereoParameters`, MMC/MultiDIC, OpenCorr, OpenCV) or entered by hand; all paths normalize to the same internal rig. Throughout, the left camera defines the world frame ($\mathbf{R} = \mathbf{I}$, $\mathbf{T} = \mathbf{0}$) and the right is expressed as $\mathbf{X}_R = \mathbf{R}\mathbf{X}_L + \mathbf{T}$.

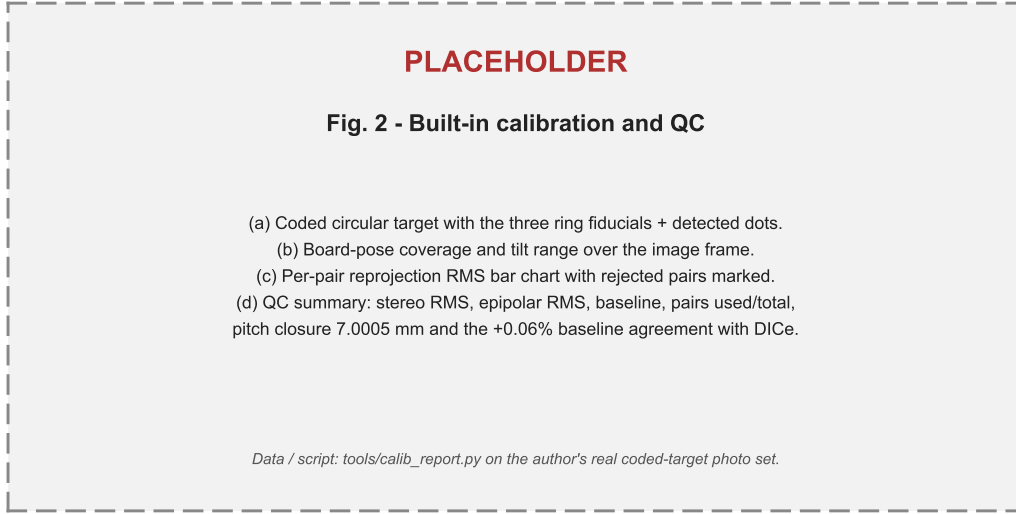


Figure 2: Built-in stereo calibration and its quality control. (a) A coded circular-dot board with the three ring fiducials and all detected dot centroids overlaid. (b) Board-pose coverage and tilt range over the image frame. (c) Per-pair reprojection RMS, with threshold-rejected pairs marked. (d) QC summary: stereo and epipolar RMS, baseline, pairs used/total, and the independent metric verification — a 7 mm pitch recovered as 7.0005 mm, and a baseline within +0.06 % of an independent DICE calibration of the same images.

2.2.2. Correspondence and 3D reconstruction

Correlation runs on the *raw* images; no rectification is performed, because resampling degrades the speckle. The calibrated geometry only seeds and bounds the search along the epipolar band [25] and supplies QC residuals; distortion is removed from point *coordinates* immediately before triangulation. Temporal tracking uses the 2D engine with the Augmented Lagrangian global step on by default, coupling the local inverse-compositional Gauss–Newton subproblems through a finite-element compatibility solve [21] over an optionally quadtree-refined mesh [22]; one switch reduces it to local DIC. Cross-camera matching is deliberately left as scattered local IC-GN, because a disparity field is projective viewpoint geometry rather than material deformation, so a displacement-compatibility regularizer does not apply — the same choice the MATLAB reference makes.

The three shipped strategies differ in which correlations they perform, and hence in how error propagates (Fig. 3b). `track_both` (default, and the strategy of the MATLAB reference) matches the cameras once on frame 1 and then tracks each independently through time: the frame-1 stereo error is a time constant that cancels to first order in the displacement, but the two temporal chains drift independently. `stereo_each_frame` tracks only

the left camera and re-matches stereo every frame, leaving one drift chain and making the reprojection residual an honest per-frame quality signal, at the cost of fresh non-cancelling stereo noise. `ref_direct` matches every image directly against left frame 1: drift-free, but it asks the hardest correlation — viewpoint difference and accumulated deformation at once — and degrades exactly when deformation is largest. Orthogonally, tracking is accumulative or incremental with a rolling reference refreshing every frame, every N frames, or on a user-supplied list; increments compose as $\mathbf{U}^k(\mathbf{X}) = \mathbf{U}^{k-1}(\mathbf{X}) + \mathbf{u}^k(\mathbf{X} + \mathbf{U}^{k-1}(\mathbf{X}))$, interpolated at the *deformed* position. The initial guess is an FFT integer search, a warm start, or user-placed *starting points* — redundant seeds, template-matched and propagated along mesh connectivity — which are the reliable choice for large inter-frame motion and for multi-region ROIs where an FFT peak-picker can pick the wrong side of a discontinuity.

Given matched, undistorted normalized coordinates and the projection matrices $\mathbf{M}_L, \mathbf{M}_R$, the world point $\mathbf{P}$ follows from the direct linear transformation [26]: each view contributes two constraints from $\mathbf{x} \times (\mathbf{M}\tilde{\mathbf{P}}) = \mathbf{0}$, and the stacked 4×4 system is solved in the total least-squares sense,

$$\mathbf{A}\tilde{\mathbf{P}} = \mathbf{0}, \quad \tilde{\mathbf{P}} = \arg \min_{\|\tilde{\mathbf{P}}\|=1} \|\mathbf{A}\tilde{\mathbf{P}}\|_2, \quad (1)$$

by singular value decomposition, followed by a reprojection-residual check. Because reconstruction returns a metric position for every node and frame, the 3D displacement is defined without reference to how correspondence was obtained,

$$\mathbf{D}^k(\mathbf{X}) = \mathbf{P}^k(\mathbf{X}) - \mathbf{P}^1(\mathbf{X}), \quad (2)$$

which is what makes the 3D layer strategy- and mode-agnostic. Invalid nodes carry NaN, which propagates unchanged to the exports; all arrays are float64 and all lengths are millimetres.

2.2.3. Surface strain, and crack awareness

Strain is a total-Lagrangian post-process on the reconstructed point cloud, evaluated at every node on the frame-1 reference surface, following the formulation validated in the MATLAB reference [20]. The virtual strain gauge is a square window of side $(n_{\text{vsg}} - 1)\Delta + 1$ pixels in the reference image (n_{vsg} the gauge size in grid steps, Δ the node spacing); the neighbourhood $\mathcal{N}(i)$ of node i is retrieved with a Chebyshev-metric k-d tree and must hold at least nine nodes with finite displacement. Because the surface is curved, gradients must be taken in its tangent plane rather than the camera frame: a plane $Z = aX + bY + c$ is fitted to the neighbourhood's reference positions, giving an orthonormal frame with $\hat{\mathbf{z}} \propto (a, b, -1)$. Reference

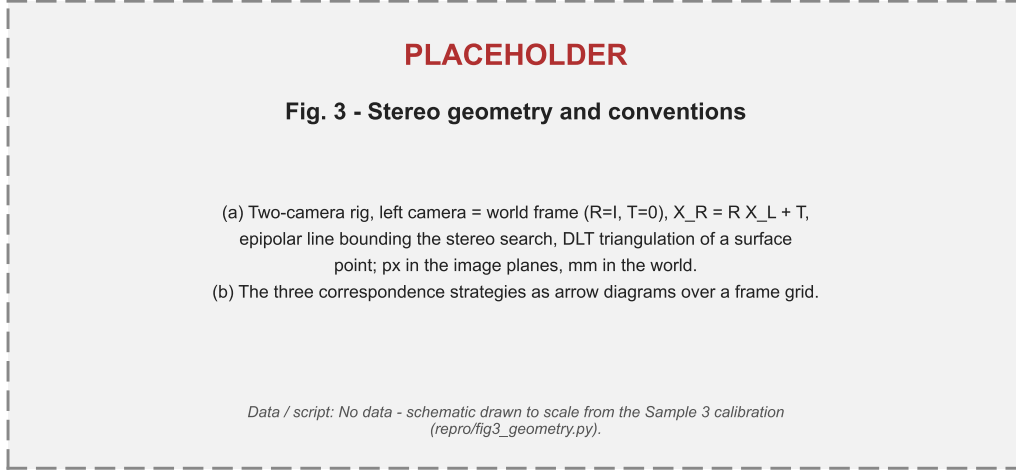


Figure 3: Measurement model and conventions. (a) The rig: the left camera defines the world frame ($\mathbf{R} = \mathbf{I}$, $\mathbf{T} = \mathbf{0}$) and the right satisfies $\mathbf{X}_R = \mathbf{R}\mathbf{X}_L + \mathbf{T}$; a surface point $\mathbf{P}$ projects to $\mathbf{x}_L$ and $\mathbf{x}_R$, its epipolar line bounds the stereo search, and Eq. (1) recovers $\mathbf{P}$ as the least-squares ray intersection. Image planes in pixels, world in millimetres. (b) The three correspondence strategies drawn over a sequence of frame pairs.

coordinates and displacements are rotated into that frame and the in-plane gradients fitted by least squares, giving a gradient matrix $\mathbf{G}$ whose out-of-plane derivative row is zeroed. The deformation gradient and Green–Lagrange strain follow as

$$\mathbf{F} = \mathbf{I} + \mathbf{G}^\top, \quad \mathbf{E} = \frac{1}{2}(\mathbf{F}^\top \mathbf{F} - \mathbf{I}), \quad (3)$$

which vanishes identically under rigid-body rotation. Principal strains, maximum shear and a von Mises invariant follow from $\mathbf{E}$, with the out-of-plane slopes retained as diagnostics. Infinitesimal ($\frac{1}{2}(\mathbf{G} + \mathbf{G}^\top)$) and Almansi ($\frac{1}{2}(\mathbf{I} - \mathbf{F}^{-\top} \mathbf{F}^{-1})$) measures are selectable, as are three reporting frames: the per-node tangent plane, the left-camera frame, or a specimen frame set by three picked points. Near a boundary the gauge is one-sided and biased, so nodes closer to it than α times the gauge radius are flagged; strain arrays stay dense and a boolean mask records what to display, so the trim can be revisited downstream.

Crack awareness reuses that machinery. With the region-of-interest cutting tools the user leaves a thin *barrier* — an excluded band — along the discontinuity, material retained on both sides; the barrier then propagates through the whole chain automatically. It cuts the correlation mesh, so no finite element bridges the crack; it is carried into the right camera by warping the left mask through the frame-1 stereo correspondence, so both

cameras are cut consistently; and it restricts each subset's correlation support to the connected component containing the subset centre, so a subset beside the crack correlates only its own face. It enters the strain gauge as a visibility test: neighbour j joins $\mathcal{N}(i)$ only if the reference-configuration segment $\overline{X_i X_j}$ does not cross the barrier, so the plane fit of Eq. (3) is never contaminated by the far face. Finally it folds into the trim mask, so the crack band is blanked rather than filled with a fabricated value, and the interactive view, the preview and the exports blank identical cells (Fig. 4). The result is a measured jump instead of a smoothed ramp; with no barrier present, every one of these steps is a verified no-op.

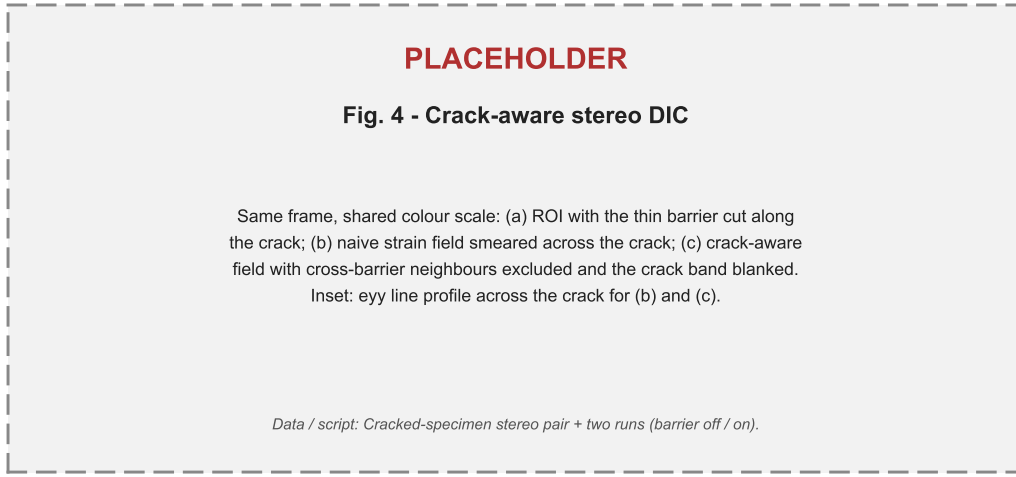


Figure 4: Crack-aware stereo DIC, all panels on a shared colour scale. (a) The region of interest with the thin barrier cut along the crack. (b) Naive result: an ordinary gauge neighbourhood averages both crack faces and reports a smooth ramp across the discontinuity. (c) Crack-aware result: no neighbour pair crosses the barrier, the crack band is blanked by the trim mask, and each face carries its own strain. Inset: ϵ_{yy} profiles perpendicular to the crack for (b) and (c), showing the jump the naive fit rounds off.

2.2.4. Independent verification of the delivered field

DIC solvers are engineered not to fail loudly. In the engine used here, local subsets that do not converge are replaced by inverse-distance-weighted neighbours, the global compatibility solve returns a field finite everywhere by construction, and incremental composition fills gaps by nearest neighbour. Each behaviour is defensible alone, but together they make a finiteness test on the output structurally always true: a frozen warm start, or a garbage increment faithfully composed, leaves a field that is finite, smooth and wrong. We observed exactly this during development, in both tracking modes.

pyALDIC-3D therefore re-verifies what it is about to ship, using a criterion the solver never optimized. After the temporal track returns, for every frame k and node $\mathbf{X}$ the reference subset $\Omega(\mathbf{X})$ is compared with the frame- k image sampled at the *reported* cumulative position $\mathbf{X} + \mathbf{U}^k(\mathbf{X})$ under a translation warp, through the zero-normalized sum of squared differences

$$\text{ZNSSD}_k(\mathbf{X}) = \sum_{\mathbf{s} \in \Omega} \left(\frac{f(\mathbf{X} + \mathbf{s}) - \bar{f}}{\Delta f} - \frac{g_k(\mathbf{X} + \mathbf{U}^k(\mathbf{X}) + \mathbf{s}) - \bar{g}_k}{\Delta g_k} \right)^2, \quad (4)$$

where $\bar{\cdot}$ and Δ denote the subset mean and the root of the summed squared deviation. The measure lies in $[0, 4]$ and is invariant to affine illumination changes; the default rejection threshold of 1.0 corresponds to a zero-normalized cross-correlation of 0.5. Nodes above it are invalidated and — crucially — *counted*: node-frames removed per camera, the per-frame validity fraction, the frame-1 stereo yield and the counts removed by each optional quality gate all reach the run log and the saved session, so a run that returns few points explains why (Fig. 5). This is an independent measurement of the delivered product, not a restatement of the solver’s residual. Because it visits every node of every frame it is a substantial part of the wall time on long sequences; the kernel is Numba-compiled and chunked, giving a $3.29\times$ speedup in v1.1.0 with bit-identical verdicts (maximum absolute difference 0 over 275,184 values).

2.2.5. Interface, sessions and distribution

The desktop application (Fig. 6), launched with `al-dic-3d gui`, has three columns: a left sidebar carrying the workflow top to bottom (paired image drop zones, calibration, workflow type, initial guess, region of interest, parameters, and an advanced section exposing the correspondence strategy); a central zoom/pan canvas showing either camera with ROI, mesh, seed and result overlays; and a right sidebar with run controls, progress, field and colormap selection, physical units, the interactive 3D view, and a filterable console log. Strain post-processing runs in its own window with independent visualization state, so gauge size, strain measure and reporting frame can be explored without recomputing correspondence. Cancelling is non-destructive: computed frames are kept and later frames left NaN.

An entire project — image paths, calibration, ROI and masks including the crack barrier, seeds, parameters, results and strain — persists to a single `.alDIC3d` file and reopens without recomputation. Exports cover NumPy `.npz`, MATLAB `.mat`, per-frame CSV, PLY point clouds and a VTU/PVD series for ParaView, plus images, MP4/GIF animations and offscreen 3D



Figure 5: Independent re-verification catching a silent solver failure. (a) Per-frame ZNSSD of the *shipped* cumulative displacement, Eq. (4), for two runs on the same data: the healthy run stays flat and low, while a run whose warm start froze rises past the rejection threshold (dashed) from the marked frame onward. (b) The displacement fields at that frame — the frozen run is finite, smooth and entirely plausible, which is why a finiteness test misses it. (c) The per-frame validity ledger reported to the user.

renders, always with a parameters JSON. Strings are translated into eight locales. The same functionality is available headless (`al-dic-3d run config.toml`, `al-dic-3d calibrate`); distribution is via PyPI, GitHub release wheels, and a standalone Windows installer needing no Python.

3. Illustrative examples

3.1. Agreement with the MATLAB reference implementation

The first requirement of a port is that it reproduces the implementation whose mathematics it adopts. On Sample 3 of the Stereo-DIC Challenge [27, 28] — a curved D-shaped coupon at 1920×1200 px — the frame $0 \rightarrow 1$ node-wise agreement with the published MATLAB 3D-Stereo-ALDIC implementation [20] is a median $|\text{difference}|$ of $1.3 \mu\text{m}$ in U , $1.0 \mu\text{m}$ in V and $6.0 \mu\text{m}$ in W , at regression slopes ≈ 1.0 ; the reconstructed shape agrees to $29.7 \mu\text{m}$ in Z (Fig. 7). The larger out-of-plane figures are expected, since triangulation amplifies image-plane error along the depth direction by roughly $1 / \sin \theta$ for stereo angle θ [29, 30]. A second gate checks the incremental machinery against a template-matching ground truth rather than against the other code, and closes at 0.38 and 0.43 px.

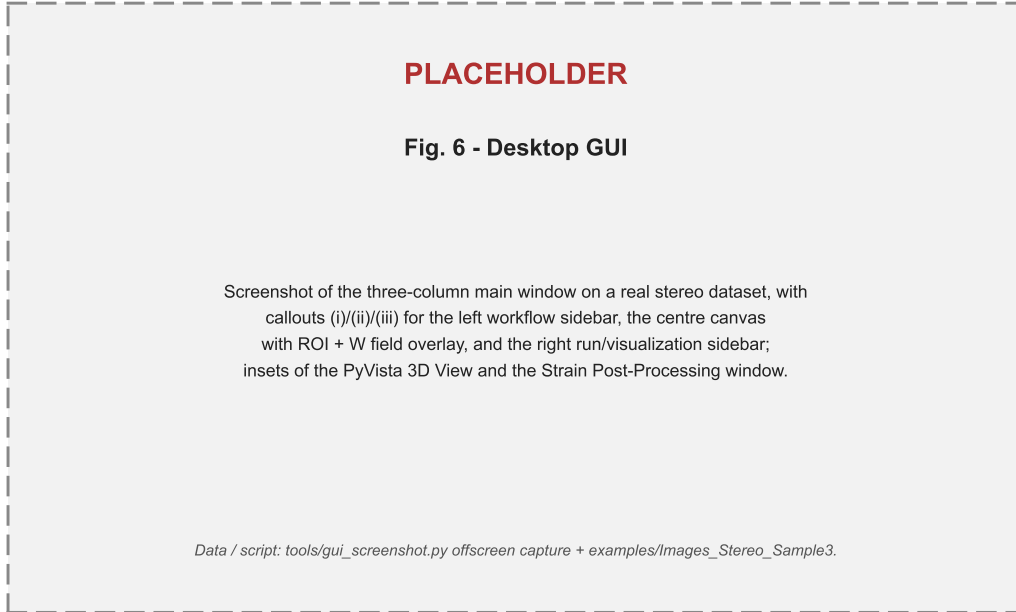


Figure 6: The pyALDIC-3D desktop application after a stereo analysis. **(i)** Left sidebar: the workflow in order — paired left/right image drop zones, calibration, workflow type, initial guess, region of interest, parameters, and the advanced panel exposing the correspondence strategy. **(ii)** Central canvas: the specimen with the region of interest and the colour-mapped out-of-plane displacement W . **(iii)** Right sidebar: run controls, progress, field and colormap selection, units, the 3D view launcher, and the console log carrying the validity ledger of Section 2.2.4. Insets: the interactive 3D view and the strain post-processing window.

3.2. Community benchmarks

On Stereo-DIC Challenge 1.0 material [27], a rigid-translation set with exact ground truth is recovered to a median $0.1\text{--}0.3\text{ }\mu\text{m}$ over 16 steps; a simulated tension sequence agrees with a MatchID analysis of the same images to $0.5\text{ }\mu\text{m}$ along the loading axis; and a real tension-to-fracture sequence runs end to end to 10.9 % strain with 98 % of nodes still valid, at a strain noise floor of $12\text{ }\mu\epsilon$. On DIC Challenge 2.0 Task 1 [31], displacements agree with DICE to a median $0.003\text{--}0.015\text{ px}$ over 11,300 points per frame, and the axial strain at the reference station is 0.269 % against the published 0.26 % anchor. The built-in calibrator was separately validated on a physical coded target: stereo reprojection RMS 0.293 px , a baseline within $+0.06\text{ }\%$ of a DICE calibration of the same photographs, and triangulated point spacing closing at $7.0005 \pm 0.0010\text{ mm}$ against a nominal 7 mm .

PLACEHOLDER

Fig. 7 - MATLAB parity and Challenge validation

(a-c) Node-wise scatter of pyALDIC-3D vs the MATLAB 3D-Stereo-ALDIC reference for U , V , W on Stereo-DIC Challenge Sample 3, with the fitted regression slope. (d) Histogram of $|diff|$ with the medians. (e) Challenge 2.0 Task 1 ϵ_{yy} vs the published anchor.

Data / script: tools/matlab_parity.py, tools/challenge_c2_task1.py, tools/challenge_s2.py.

Figure 7: Validation. (a–c) Node-wise comparison against the MATLAB 3D-Stereo-ALDIC reference [20] on Stereo-DIC Challenge Sample 3, frame $0 \rightarrow 1$, for U , V and W , with the 1:1 line and fitted slope. (d) Histograms of the node-wise absolute difference (medians 1.3, 1.0, 6.0 μm). (e) DIC Challenge 2.0 Task 1: recovered axial strain against the published anchor.

3.3. Scale, accuracy over long sequences, and performance

Experimental campaigns produce long sequences at full sensor resolution, where naive implementations exhaust memory or accumulate drift. Two stress runs on synthetic data with known ground truth characterize the limits (Table 4, Fig. 8). A 150-frame sequence of 12 Mpx pairs (27,170 nodes) completes in 58.9 min at a peak resident set of 5.68 GB, close to the 4.63 GB projected beforehand by the fail-fast memory check; images are streamed lazily rather than held, so memory is governed by the mesh and working set, not by sequence length. A 400-frame sequence of 5 Mpx pairs (11,466 nodes) completes in 48.4 min at a median 1.02 s per frame with 100 % node validity, and — the point of the test — the end-frame accuracy against ground truth is 0.011 px in the image plane and 0.0039 mm in 3D, with no drift. On top of NumPy and SciPy [32, 33], the two heaviest kernels are compiled by Numba [34]: the strain kernel is $6.5\text{--}19\times$ faster than the array reference (agreement below 10^{-9}), the verification kernel $3.29\times$ faster with bit-identical verdicts.

	150 frames $\times$ 12 Mpx	400 frames $\times$ 5 Mpx
Nodes	27,170	11,466
Wall time	58.9 min	48.4 min
Median tracking time / frame	—	1.02 s
Peak resident memory	5.68 GB	3.06 GB
Node validity	100 %	100 %
End-frame error, image plane	0.010 px	0.011 px
End-frame error, 3D point	0.002 mm	0.0039 mm

Table 4: Scale tests on synthetic stereo sequences with known ground truth. “End-frame error” is measured against the synthetic ground truth at the last frame and shows no growth with sequence length.

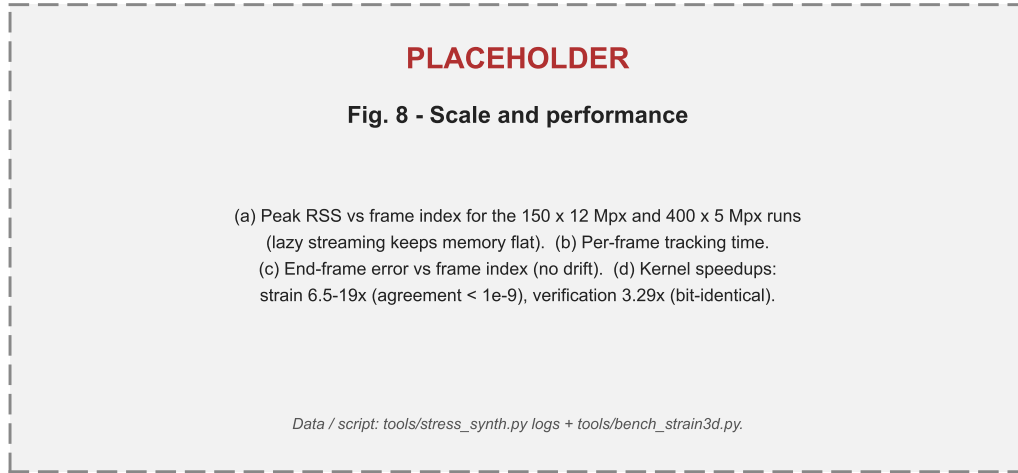


Figure 8: Scale and performance. (a) Resident memory against frame index for the two runs of Table 4: lazy streaming keeps memory flat rather than growing with sequence length. (b) Per-frame tracking time over the 400-frame run, showing no degradation. (c) Error against the synthetic ground truth versus frame index, demonstrating the absence of drift. (d) Numba kernel speedups: $6.5\text{--}19\times$ (strain) and $3.29\times$ (verification).

4. Impact and conclusions

Removing the barriers to stereo-DIC. Stereo-DIC has been practically gated behind either a commercial license or a MATLAB license plus a chain of separate tools. pyALDIC-3D delivers the complete chain — calibration, correspondence, reconstruction, finite surface strain, visualization and export — as one BSD-3-Clause application that installs with a single command on Windows, macOS and Linux, or from a double-clickable installer for users with no Python at all. Because the core is Qt-free, the identical pipeline runs headless, so exploratory work and batch reprocessing share one implementation and one result format.

A position on trustworthy DIC software. The verification gate of Section 2.2.4 is a methodological point rather than a feature. Full-field measurement software returns dense arrays of plausible numbers whether or not the underlying correlations succeeded, and the usual defences — correlation residual, convergence flag, finiteness — are all statements the solver makes about itself. We argue that a measurement tool should additionally re-measure its own deliverable against a criterion it did not optimize, and account for every point it discards. Doing so cost one pass over the data and caught failures that would otherwise have been published; we recommend the practice to other DIC implementations.

Standards alignment and provenance. Parameter naming, region-of-interest conventions, calibration QC reporting and strain-gauge terminology follow the iDICs *Good Practices Guide* [5, 35]. The underlying algorithms are community-validated: AL-DIC [21] was assessed in DIC Challenge 2.0 [31] and extended to volumetric [36] and stereo [20] settings, and this implementation is checked directly against that stereo reference.

Conclusions and roadmap. pyALDIC-3D is an open-source, cross-platform, GUI-based stereo-DIC application spanning calibration through 3D surface strain, with pluggable correspondence strategies, crack-aware handling of discontinuous fields and independent verification of its own output. It agrees with the published MATLAB reference to $1.3/1.0/6.0\ \mu\text{m}$ in $U/V/W$ and with community benchmarks at the sub-pixel level, and sustains 400-frame sequences without drift. The roadmap targets N -camera rigs, a quality-driven adaptive correspondence strategy that re-anchors on demand and rescues lost points, and GPU acceleration. Source code, documentation and worked examples are openly available at <https://github.com/zachtong/pyALDIC-3D>.

CRedit authorship contribution statement

Zixiang Tong: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing. **Jin Yang:** Conceptualization, Methodology, Supervision, Resources, Funding acquisition, Project administration, Writing – review & editing.

Conflict of interest

The authors declare no conflict of interest. There has been no significant financial support for this work that could have influenced its outcome.

Data availability

Runnable worked examples, their configuration files and the scripts that regenerate every reported figure are distributed with the source repository and archived on Zenodo [1]. The Stereo-DIC Challenge and DIC Challenge 2.0 materials used for validation are distributed by the International Digital Image Correlation Society [28]. Experimental images acquired by the authors are released under the same license as the code.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic Claude to polish manuscript text, to generate schematic explanatory figures via matplotlib code, and to format LaTeX tables. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Acknowledgements

Z.T. thanks the University Graduate Continuing Fellowship, the Graduate Excellence Fellowship, and the Professional Development Award from the University of Texas at Austin Cockrell School of Engineering. J.Y. gratefully acknowledges support from the U.S. National Science Foundation (NSF) under Grants No. 2232428, 2441460, and 2452029, and the Pilot Seed Gift from the Semiconductor Research Corporation.

References

- [1] Z. Tong, J. Yang, pyALDIC-3D: stereo / 3D digital image correlation built on the pyALDIC platform, Zenodo software archive, version v1.1.0, <https://doi.org/10.5281/zenodo.21696564> (2026). doi:10.5281/zenodo.21696564.
- [2] M. A. Sutton, J.-J. Orteu, H. W. Schreier, Image Correlation for Shape, Motion and Deformation Measurements: Basic Concepts, Theory and Applications, Springer, Boston, MA, 2009. doi:10.1007/978-0-387-78747-3.
- [3] B. Pan, Digital image correlation for surface deformation measurement: historical developments, recent advances and future goals, Measurement Science and Technology 29 (8) (2018) 082001. doi:10.1088/1361-6501/aac55b.
- [4] B. Pan, L. Yu, Q. Zhang, Review of single-camera stereo-digital image correlation techniques for full-field 3D shape and deformation measurement, Science China Technological Sciences 61 (1) (2017) 2–20. doi:10.1007/s11431-017-9090-x.
- [5] E. M. C. Jones, M. A. Iadicola (Eds.), A Good Practices Guide for Digital Image Correlation, International Digital Image Correlation Society, 2018. doi:10.32720/idics/gpg.ed1.
- [6] P. Reu, Stereo-rig design: creating the stereo-rig layout — Part 1, Experimental Techniques 36 (5) (2012) 3–4. doi:10.1111/j.1747-1567.2012.00871.x.
- [7] P. Reu, Calibration: care and feeding of a stereo-rig, Experimental Techniques 38 (3) (2014) 1–2. doi:10.1111/ext.12083.
- [8] D. Solav, K. M. Moerman, A. M. Jaeger, K. Genovese, H. M. Herr, MultiDIC: an open-source toolbox for multi-view 3D digital image correlation, IEEE Access 6 (2018) 30520–30535. doi:10.1109/ACCESS.2018.2843725.
- [9] D. Solav, A. Silverstein, DuoDIC: 3D digital image correlation in MATLAB, Journal of Open Source Software 7 (74) (2022) 4279. doi:10.21105/joss.04279.
- [10] J. Blaber, B. Adair, A. Antoniou, Ncorr: open-source 2D digital image correlation MATLAB software, Experimental Mechanics 55 (6) (2015) 1105–1122. doi:10.1007/s11340-015-0009-1.

- [11] D. Atkinson, T. H. Becker, Stereo digital image correlation in MATLAB, *Applied Sciences* 11 (11) (2021) 4904. doi:10.3390/app11114904.
- [12] D. Z. Turner, Digital image correlation engine (DICE) reference manual, Tech. Rep. SAND2015-10606 O, Sandia National Laboratories (2015).
- [13] D. Z. Turner, An overview of the stereo correlation and triangulation formulations used in DICE, Tech. rep., Sandia National Laboratories (2017). doi:10.2172/1367484.
- [14] Z. Jiang, OpenCorr: an open source library for research and development of digital image correlation, *Optics and Lasers in Engineering* 165 (2023) 107566. doi:10.1016/j.optlaseng.2023.107566.
- [15] J. C. A. de Deus Filho, L. C. d. S. Nunes, J. M. C. Xavier, iCorrVision-3D: an integrated Python-based open-source digital image correlation software for in-plane and out-of-plane measurements (Part 2), *SoftwareX* 19 (2022) 101132. doi:10.1016/j.softx.2022.101132.
- [16] Correlated Solutions, Inc., VIC-3D digital image correlation system, <https://www.correlatedsolutions.com/vic-3d>, commercial software; accessed 2026-07-31 (2026).
- [17] MatchID NV, MatchID stereo digital image correlation, <https://www.matchid.eu/>, commercial software; accessed 2026-07-31 (2026).
- [18] ZEISS / GOM, ARAMIS 3D optical strain and displacement analysis, <https://www.gom.com/en/products/zeiss-aramis>, commercial software; accessed 2026-07-31 (2026).
- [19] Z. Tong, J. Yang, pyALDIC: a Python implementation of augmented Lagrangian digital image correlation with a GUI, adaptive meshing, and mask-aware subset splitting, preprint, under review (2026). arXiv:2607.22755, doi:10.48550/arXiv.2607.22755.
- [20] Z. Tong, D. Frolkin, H. Shi, T. LaRue, M. K. Rausch, Y. Zhang, V. Knight, J. Yang, 3D stereo adaptive mesh augmented Lagrangian digital image correlation, *Experimental Mechanics* 65 (9) (2025) 1387–1411. doi:10.1007/s11340-025-01225-7.
- [21] J. Yang, K. Bhattacharya, Augmented Lagrangian digital image correlation, *Experimental Mechanics* 59 (2) (2019) 187–205. doi:10.1007/s11340-018-00457-0.

- [22] J. Yang, V. Rubino, Z. Ma, J. Tao, Y. Yin, A. McGhee, W. Pan, C. Franck, SpatioTemporally Adaptive Quadtree mesh (STAQ) digital image correlation for resolving large deformations around complex geometries and discontinuities, *Experimental Mechanics* 62 (7) (2022) 1191–1215. doi:10.1007/s11340-022-00872-4.
- [23] Z. Zhang, A flexible new technique for camera calibration, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22 (11) (2000) 1330–1334. doi:10.1109/34.888718.
- [24] G. Bradski, The OpenCV library, *Dr. Dobb’s Journal of Software Tools* (2000).
- [25] R. Hartley, A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd Edition, Cambridge University Press, 2004. doi:10.1017/CB09780511811685.
- [26] Y. I. Abdel-Aziz, H. M. Karara, Direct linear transformation from comparator coordinates into object space coordinates in close-range photogrammetry, *Photogrammetric Engineering & Remote Sensing* 81 (2) (2015) 103–107, reprint of the 1971 ASP Symposium paper. doi:10.14358/PERS.81.2.103.
- [27] W. Ahmad, J. Helm, S. Bossuyt, P. Reu, D. Turner, L. K. Luan, P. Lava, T. Siebert, M. Simonsen, Stereo-DIC challenge 1.0 — rigid body motion of a complex shape, *Experimental Mechanics* 64 (7) (2024) 1073–1106. doi:10.1007/s11340-024-01077-7.
- [28] International Digital Image Correlation Society, DIC Challenge, <https://idics.org/challenge/>, benchmark image sets and ground truth; accessed 2026-07-31 (2026).
- [29] P. Reu, Stereo-rig design: stereo-angle selection — Part 4, *Experimental Techniques* 37 (2) (2013) 1–2. doi:10.1111/ext.12006.
- [30] R. Balcaen, P. L. Reu, P. Lava, D. Debruyne, Stereo-DIC uncertainty quantification based on simulated images, *Experimental Mechanics* 57 (6) (2017) 939–951. doi:10.1007/s11340-017-0288-9.
- [31] P. L. Reu, B. Blaysat, E. Andó, K. Bhattacharya, C. Couture, V. Couty, D. Deb, S. S. Fayad, M. A. Iadicola, S. Jaminion, M. Klein, A. K. Landauer, P. Lava, M. Liu, L. K. Luan, S. N. Olufsen, J. Réthoré, E. Roubin, D. T. Seidl, T. Siebert, O. Stamati, E. Toussaint, D. Turner, C. S. R. Vemulapati, T. Weikert, J. F. Witz, O. Witzel, J. Yang, DIC

- challenge 2.0: developing images and guidelines for evaluating accuracy and resolution of 2D analyses: focus on the metrological efficiency indicator, *Experimental Mechanics* 62 (4) (2022) 639–654. doi:10.1007/s11340-021-00806-6.
- [32] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, et al., Array programming with NumPy, *Nature* 585 (7825) (2020) 357–362. doi:10.1038/s41586-020-2649-2.
 - [33] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, et al., SciPy 1.0: fundamental algorithms for scientific computing in Python, *Nature Methods* 17 (3) (2020) 261–272. doi:10.1038/s41592-019-0686-2.
 - [34] S. K. Lam, A. Pitrou, S. Seibert, Numba: a LLVM-based Python JIT compiler, in: *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, ACM, 2015, pp. 1–6. doi:10.1145/2833157.2833162.
 - [35] E. M. C. Jones, M. A. Iadicola (Eds.), *A Good Practices Guide for Digital Image Correlation*, Edition 2, International Digital Image Correlation Society, 2025. doi:10.32720/idics/gpg.ed2.
 - [36] J. Yang, L. Hazlett, A. K. Landauer, C. Franck, Augmented Lagrangian digital volume correlation (ALDVC), *Experimental Mechanics* 60 (9) (2020) 1205–1223. doi:10.1007/s11340-020-00607-3.